

FISSER HandBeam 10D

Bitácora Técnica de Desarrollo

ESP32-2432S032 · ST7789 · GT911 · LVGL 8.3.9 · EEZ Studio

Versión v3.0	Fecha Marzo 2026	Entradas 36
------------------------	----------------------------	-----------------------

Tabla de Contenidos

Sección	Contenido	Pág.
I	Introducción y Hardware	3
II	Instalación del Entorno: Arduino IDE	4
III	Entorno Avanzado: VS Code + Arduino Extension	6
IV	Librerías Utilizadas	9
V	Configuración de Display y Touch	10
VI	Integración LVGL + EEZ Studio	12
VII	Pantallas Implementadas	16
VIII	Variables Globales	21
IX	Referencia de Expresiones EEZ Flow	23
X	Limitaciones Conocidas	24
XI	Bitácora de Entradas de Desarrollo	25
XII	Estado del Proyecto	40

I — Introducción y Hardware

Este documento registra el proceso completo de desarrollo de la interfaz gráfica para el dispositivo médico de terapia láser Fisser HandBeam 10D. La UI fue construida con EEZ Studio (diseño visual) + LVGL 8.3.9 (motor gráfico) + Arduino/ESP32 (firmware).

Hardware del dispositivo

Componente	Especificación	Notas
MCU	ESP32 Dev Module (ESP-WROOM-32)	Dual core 240 MHz, 4MB Flash
PCB	ESP32-2432S032	Placa all-in-one con pantalla integrada
Display	ST7789 IPS 3.2" 240×320	Interface SPI, tipo IPS, excelente ángulo de visión
Touch	GT911 Capacitivo (I2C)	Hasta 5 puntos, sin necesidad de calibración
Backlight	Pin GPIO 27	Control por PWM o GPIO digital

Mapa de pines

Función	Pin ESP32	Señal
DC (Data/Command)	GPIO 2	SPI Display
CS (Chip Select)	GPIO 15	SPI Display
SCK (Clock)	GPIO 14	SPI Display
MOSI	GPIO 13	SPI Display
Backlight	GPIO 27	Display BL
RST Display	-1	No conectado
SDA Touch	GPIO 33	I2C GT911
SCL Touch	GPIO 32	I2C GT911
RST Touch	GPIO 25	I2C GT911
INT Touch	-1	No utilizado

II — Instalación del Entorno: Arduino IDE

Arduino IDE es el entorno base requerido incluso cuando se usa VS Code como editor principal. El compilador y las herramientas de flash del ESP32 son gestionados por Arduino IDE y sus paquetes.

1. Instalar Arduino IDE

Instalar la versión 1.8.19 o superior. El proyecto fue desarrollado con Arduino IDE 1.8.19.

Descarga: <https://downloads.arduino.cc/arduino-1.8.19-windows.exe>

2. Agregar soporte para ESP32

Abrir Arduino IDE y seguir estos pasos para instalar el paquete de placas ESP32 de Espressif:

1. Abrir menú File → Preferences.
2. En el campo "Additional Boards Manager URLs" pegar la siguiente URL:
`https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package\_esp32\_index.json`
3. Hacer clic en OK para guardar.
4. Ir a Tools → Board → Boards Manager.
5. Buscar "esp32" e instalar "esp32 by Espressif Systems".
6. Una vez instalado, seleccionar Tools → Board → ESP32 Arduino → ESP32 Dev Module.
7. Configurar en Tools: Upload Speed: 921600, Flash Size: 4MB, Programmer: Esptool.

⚠ Versión instalada y probada: ESP32 Core 3.3.5. Otros números de versión pueden presentar incompatibilidades con las APIs utilizadas.

3. Instalar librerías desde el Library Manager

Abrir Tools → Manage Libraries y buscar e instalar las siguientes:

Librería	Versión usada	Buscador	Notas
GFX Library for Arduino	1.6.4	"Arduino_GFX" o "GFX Library"	Driver de display. Buscar por 'Moon On Our Nation'
lvgl	8.3.9	"lvgl"	Motor gráfico. Instalar solo LVGL, no lv_examples
TAMC_GT911	1.0.2	"GT911" o "TAMC"	Driver touch capacitivo. Aparece como TAMC_GT911

✓ La librería TAMC_GT911 v1.0.2 SÍ aparece en el gestor de librerías de Arduino IDE. Buscarla por el término "GT911" o "TAMC".

4. Configurar lv_conf.h

Después de instalar LVGL, el archivo lv_conf.h debe ser copiado y configurado manualmente:

8. Localizar el archivo plantilla en:

```
C:\Users\DANNY\Documents\Arduino\libraries\lvgl\lv_conf_template.h
```

9. Copiar el archivo como lv_conf.h en el directorio de librerías:

```
C:\Users\DANNY\Documents\Arduino\libraries\lv_conf.h
```

10. Editar el archivo y cambiar `#if 0` por `#if 1` al inicio.

11. Habilitar las fuentes requeridas por el proyecto:

```
#define LV_FONT_MONTSEERRAT_10 1
#define LV_FONT_MONTSEERRAT_12 1
#define LV_FONT_MONTSEERRAT_18 1
#define LV_FONT_MONTSEERRAT_22 1
#define LV_FONT_MONTSEERRAT_34 1
#define LV_USE_QRCODE 1
```

12. Habilitar el widget QR:

⚠ El archivo lv_conf.h NO debe contener `#include <stdint.h>` en el nivel global. Si lo tiene, eliminar esa línea — causa errores al compilar archivos .S de ensamblador ARM.

III — Entorno Avanzado: VS Code + Arduino Extension

¿Por qué VS Code en lugar de solo Arduino IDE?

Arduino IDE 1.8.x, aunque funcional, presenta limitaciones importantes para un proyecto de la complejidad del HandBeam 10D (múltiples archivos .c/.h, lógica de Flow generada, ~2000 líneas de código exportado). La combinación VS Code + Arduino Extension resuelve estos problemas:

Aspecto	Solo Arduino IDE	VS Code + Arduino
Tiempo de compilación	Largo (recompila TODO cada vez)	Reducido: solo recompila cambios
Autocompletado	Básico / inexistente	IntelliSense completo con sugerencias en tiempo real
Detección de errores	Solo al compilar	Subrayado rojo instantáneo antes de compilar
Navegación de código	Limitada	Ir a definición, buscar referencias, peek definition
Manejo de archivos	Un archivo visible	Explorador lateral con toda la estructura del proyecto
Git integration	Ninguna	Control de versiones integrado
Múltiples cursores	No	Sí — edición simultánea en varias líneas
Tamaño del editor	Fijo	Totalmente personalizable, multi-ventana
Terminal integrado	No	Sí — sin salir del editor

Importante: VS Code NO reemplaza a Arduino IDE. Sigue siendo necesario tener Arduino IDE instalado. VS Code usa el compilador (xtensa-esp32-elf-g++) y el uploader (esptool.py) que Arduino IDE instaló. VS Code actúa como un front-end más poderoso para las mismas herramientas.

Instalación de VS Code y la extensión Arduino

1. Descargar VS Code desde <https://code.visualstudio.com/> e instalarlo.
2. Abrir VS Code, ir a la sección de extensiones (Ctrl+Shift+X).
3. Buscar "Arduino" e instalar la extensión oficial de Microsoft: "Arduino" (id: vsciot-vscode.vscode-arduino).
4. Reiniciar VS Code después de la instalación.
5. Abrir la carpeta del proyecto: File → Open Folder → seleccionar la carpeta test/.

Archivos de configuración del proyecto (.vscode/)

La carpeta .vscode/ contiene los archivos de configuración que le dicen a VS Code cómo compilar y conectarse al ESP32. Estos archivos son específicos del proyecto y deben mantenerse bajo control de versiones.

arduino.json — Configuración de la placa y puerto de comunicación:

```
{
  "board": "esp32:esp32:esp32",
  "sketch": "test.ino",
  "port": "COM3",
  "output": "../build",
  "configuration": "PartitionScheme=default,CPUFreq=240,
    FlashMode=dio,FlashFreq=80,FlashSize=4M,
    UploadSpeed=921600,LoopCore=1,EventsCore=1,
    DebugLevel=none,EraseFlash=none"
}
```

⚠ El campo "port" debe coincidir con el puerto COM asignado al ESP32 en tu sistema. Verificar en Administrador de dispositivos de Windows.

c_cpp_properties.json — Configuración de IntelliSense (rutas de includes y defines):

Este archivo es el más crítico para el funcionamiento de IntelliSense. Le dice a VS Code dónde están los archivos de cabecera de todas las librerías, el SDK de ESP32 y el compilador xtensa. Sin este archivo, VS Code no puede resolver los #include y muestra líneas rojas falsas en todo el código.

Las rutas principales configuradas en el proyecto:

```
Compilador: ...\\packages\\esp32\\tools\\esp-x32\\2511\\bin\\xtensa-esp32-elf-g++

Includes principales:
- ...\\packages\\esp32\\hardware\\esp32\\3.3.5\\cores\\esp32
- ...\\packages\\esp32\\hardware\\esp32\\3.3.5\\variants\\esp32
- ...\\Arduino\\libraries\\GFX_Library_for_Arduino\\src
- ...\\Arduino\\libraries\\lvgl\\src
- ...\\Arduino\\libraries\\Touch_GT911
- ...\\packages\\esp32\\tools\\esp32-arduino-libs\\...
```

settings.json — Configuración del comportamiento del editor:

```
{
  "C_Cpp.errorSquiggles": "enabled"
}
```

Compilar y subir desde VS Code

Con VS Code configurado, los comandos de compilación y upload están disponibles en la barra de estado inferior o mediante la paleta de comandos (Ctrl+Shift+P):

Acción	Shortcut / Comando	Equivalente Arduino IDE
Verificar/Compilar	Ctrl+Alt+R o "Arduino: Verify"	Botón ✓ (Verify)
Compilar y subir	Ctrl+Alt+U o "Arduino: Upload"	Botón → (Upload)
Abrir Monitor Serie	Ctrl+Alt+M o "Arduino: Open Serial Monitor"	Tools → Serial Monitor
Seleccionar placa	"Arduino: Change Board Type"	Tools → Board
Seleccionar puerto	"Arduino: Select Serial Port"	Tools → Port

La carpeta de salida del build está configurada como ../build (relativa al sketch). Esto mantiene los archivos temporales de compilación fuera de la carpeta del proyecto y acelera compilaciones posteriores, ya que los archivos objeto (.o) que no cambiaron se reutilizan.

IV — Librerías Utilizadas

Las siguientes versiones fueron probadas y verificadas en hardware real (ESP32-2432S032):

Librería	Versión	Fuente	Repositorio / Notas
GFX Library for Arduino	1.6.4	Library Manager	github.com/moononournation/Arduino_GFX
lvgl	8.3.9	Library Manager	github.com/lvgl/lvgl — Solo instalar 'lvgl', NO 'lv_examples'
TAMC_GT911	1.0.2	Library Manager	Driver touch capacitivo GT911 por TAMCTec
ESP32 Arduino Core	3.3.5	Boards Manager	espressif/arduino-esp32 — package_esp32_index.json
EEZ Studio	0.17.x	eezstudio.io	IDE de diseño de UI. Genera código LVGL 8.x

Repositorio de referencia del fabricante de la placa ESP32-2432S032 (encontrado en GitHub):

https://github.com/kpeeem/3.2inch_ESP32-2432S032

Este repositorio contiene demos, documentación, esquemáticos y ejemplos de LVGL + GT911. Fue la fuente principal de referencia para la configuración inicial del hardware.

V — Configuración de Display y Touch

Inicialización del display ST7789 (LVGL 9.x API)

```
// Display init
lv_display_t *disp = lv_display_create(240, 320);
lv_display_set_flush_cb(disp, my_disp_flush);
lv_display_set_buffers(disp, draw_buf, NULL, DRAW_BUF_SIZE,
                      LV_DISPLAY_RENDER_MODE_PARTIAL);
lv_display_set_color_format(disp, LV_COLOR_FORMAT_RGB565);

// Flush callback
void my_disp_flush(lv_display_t *disp, const lv_area_t *area, uint8_t *px_map) {
    int32_t w = area->x2 - area->x1 + 1;
    int32_t h = area->y2 - area->y1 + 1;
    gfx->draw16bitRGBBitmap(area->x1, area->y1, (uint16_t*)px_map, w, h);
    lv_display_flush_ready(disp);
}
```

Inicialización del touch GT911 (Portrait Mode)

El GT911 reporta coordenadas en orientación landscape (320×240 internamente). Para usar en portrait mode (240×320), las coordenadas deben transformarse:

```
Touch_GT911 ts(SDA=33, SCL=32, INT=-1, RST=25, w=320, h=240);

void touch_init() {
    Wire.begin(33, 32);
    ts.begin();
    ts.setRotation(ROTATION_LEFT);
}

// Transformación portrait (rotation=0):
display_x = map(raw_y, 0, 240, 0, 239);
display_y = map(raw_x, 0, 320, 319, 0);
```

Control del Backlight (ESP32 Core 3.x)

✗ API antigua (Core 2.x) — NO usar:

```
ledcSetup(0, 2000, 8); ledcAttachPin(TFT_BL, 0); ledcWrite(0, 255);
```

✓ API correcta (Core 3.x) — usar:

```
// Opción 1: GPIO digital (100% brillo)
pinMode(TFT_BL, OUTPUT);
digitalWrite(TFT_BL, HIGH);

// Opción 2: PWM (brillo variable)
ledcAttach(TFT_BL, 2000, 8);
ledcWrite(TFT_BL, 255); // Ahora el primer arg es el PIN, no el canal
```

VI — Integración LVGL + EEZ Studio

Flujo de trabajo

Paso	Herramienta	Descripción
1. Diseño visual	EEZ Studio	Diseñar pantallas arrastrando widgets, definir variables, crear lógica de Flow
2. Exportar código	EEZ Studio	File → Export → genera archivos .c y .h para LVGL 8
3. Copiar archivos	Windows Explorer	Copiar screens.c y eez-flow.cpp a la carpeta src/ del proyecto
4. Aplicar fix manual	VS Code (Ctrl+H)	Corregir el cast lv_part_t en eez-flow.cpp (ver abajo)
5. Compilar	VS Code / Arduino IDE	Verificar y subir al ESP32

Fix obligatorio en eez-flow.cpp tras cada exportación

EEZ Studio genera código para LVGL 8, pero el proyecto usa LVGL 9. La función `lv_obj_get_style_opa()` cambió su firma. Aplicar estos dos reemplazos con Ctrl+H en VS Code después de cada exportación:

Buscar:

```
lv_obj_get_style_opa((lv_obj_t *)a->user_data, 0)
```

Reemplazar con:

```
lv_obj_get_style_opa((lv_obj_t *)a->user_data, (lv_part_t)LV_PART_MAIN)
```

Buscar:

```
lv_obj_get_style_opa(obj, 0)
```

Reemplazar con:

```
lv_obj_get_style_opa(obj, (lv_part_t)LV_PART_MAIN)
```

¿Qué archivos copiar tras cada exportación?

Archivo	¿Copiar?	Condición
eez-flow.cpp	SIEMPRE	Contiene la lógica de Flow. Requiere fix lv_part_t después
screens.c	Si hubo cambios de widgets	Si solo cambió la lógica (Flow), este archivo no cambia
screens.h	Raramente	Solo si se agregaron/quitaron pantallas
styles.c / .h	Solo si cambiaron estilos	Normalmente no cambia

Archivo	¿Copiar?	Condición
images.c / .h	Solo si cambiaron imágenes	Normalmente no cambia
ui.c / .h	Raramente	Solo si cambió la estructura de pantallas
vars.h	Solo si se agregaron variables	Normalmente no cambia una vez definidas
lv_conf.h	NUNCA	EEZ Studio no lo toca. Se gestiona manualmente

Estructura del proyecto

```

test/
├── test.ino                ← Código principal (setup, loop, callbacks)
├── compile_commands.json  ← IntelliSense para VS Code
├── Readme.txt             ← Documentación técnica
├── .vscode/               ← Configuración VS Code
│   ├── arduino.json       ← Placa, puerto, configuración
│   ├── c_cpp_properties.json ← IntelliSense / includes
│   └── settings.json      ← Comportamiento del editor
└── src/                   ← Archivos generados por EEZ Studio
    ├── ui.h / ui.c        ← ui_init(), ui_tick()
    ├── screens.h / .c     ← Widgets de todas las pantallas
    ├── eez-flow.cpp       ← Lógica de Flow (eventos, acciones)
    ├── styles.h / .c      ← Estilos visuales
    ├── images.h / .c      ← Imágenes en arrays C
    ├── vars.h             ← Variables globales
    └── actions.h          ← Prototipos de acciones

```

VII — Pantallas Implementadas

Pantalla	Nombre en EEZ	Descripción
1	SplashScreen	Boot con barra de progreso animada y logotipo
2	login_SCREEN	PIN de 4 dígitos con teclado numérico y toggle de visibilidad
3	MENU	Pantalla principal: LEDs, temporizador, cálculo de dosis, batería
4	SETTINGS	Configuración general con código QR
5	CAMBIO_PASSWORD	Cambio seguro del PIN de acceso

Detalles de implementación por pantalla

login_SCREEN — Toggle de visibilidad del PIN:

LVGL no permite cambiar el modo password de un TextArea en tiempo de ejecución mediante expresiones. Solución: dos TextAreas superpuestos en la misma posición:

- textarea_password: modo password ON, Hidden = !eyeOpen
- textarea_plain: modo password OFF, Hidden = eyeOpen
- Ambos ligados a la variable output_keyboard.
- Botón ojo CLICKED: SetVariable eyeOpen ← !eyeOpen

login_SCREEN — Notificación (reemplaza MessageBox roto):

El widget MessageBox de EEZ Studio está marcado como 'work in progress'. En hardware real, al presionar OK el overlay modal no se destruye correctamente, congelando toda la pantalla. Solución: sistema de Container + Label:

- Container CONTAINER_NOTIFICATION_TEXT con Label hijo.
- Container FLAGS Hidden Expression: !show_notification
- Label Text Expression: notification_text, alineación CENTER.
- Flow al validar PIN incorrecto: notif_text ← 'PIN Incorrecto' → show_notification ← true → Delay 1500ms → show_notification ← false → output_keyboard ← "

MENU — Indicador de batería:

6 labels LED con iconos, cada uno con Hidden Expression basada en la variable battery_state (0=vacío, 1=25%, 2=50%, 3=75%, 4=lleno, 5=cargando).

MENU — LEDs y checkboxes:

El color de los LEDs se asigna via SPECIFIC → Color → Expression (NO via STYLE, ya que STYLE aplica un sistema de brillo que distorsiona los colores rojos convirtiéndolos en cafés):

- LED Rojo: check_R ? 0xff0000 : 0x000000
- LED Violeta: check_V ? 0x8800ff : 0x000000

- LED IR: check_IR ? 0x8b0000 : 0x000000

MENU — Temporizador countdown:

Implementado como User Action countdown_tick con patrón Label IN/OUT:

```
Label IN: tick
  IsTrue: is_running && treatment_time > 0
    No → SetVariable is_running ← false → End
    Yes → Delay 1000ms
        SetVariable treatment_time ← treatment_time - 1
  IsTrue: treatment_time == 0
    Yes → SetVariable is_running ← false → End
    No → Label OUT: tick (vuelve al inicio)
```

CAMBIO_PASSWORD — Teclado contextual:

Tres TextAreas (todos en modo password). ButtonMatrix oculto por default. Cada TextArea al recibir foco: active_field ← N, keyboard_visible ← true. Al perder foco: keyboard_visible ← false. La ButtonMatrix escribe en el campo activo via SwitchCase sobre active_field.

VIII — Variables Globales (20 variables)

Variable	Tipo	Default	Propósito
output_keyboard	string	""	Salida del teclado numérico, compartida por todas las pantallas
eyeOpen	boolean	true	Estado del toggle de visibilidad del PIN
stored_password	string	"1234"	PIN real almacenado. NUNCA se muestra en UI.
pass_current	string	""	Campo de contraseña actual en CAMBIO_PASSWORD
pass_new	string	""	Campo de nueva contraseña en CAMBIO_PASSWORD
pass_confirm	string	""	Campo de confirmación en CAMBIO_PASSWORD
battery_state	integer	0	0=vacío, 1=25%, 2=50%, 3=75%, 4=100%, 5=cargando
check_R	boolean	false	Estado del checkbox LED Rojo
check_V	boolean	false	Estado del checkbox LED Violeta
check_IR	boolean	false	Estado del checkbox LED Infrarrojo
treatment_time	integer	0	Tiempo de tratamiento en segundos
is_running	boolean	false	True cuando el countdown está activo
dosis_p_c	integer	0	Dosis puntual ×100 (para mostrar decimales como enteros)
dosis_t_c	integer	0	Dosis total ×100
keyboard_visible	boolean	false	Visibilidad del teclado en CAMBIO_PASSWORD
active_field	integer	0	Campo activo en CAMBIO_PASSWORD: 1=actual, 2=nueva, 3=confirmar
show_notification	boolean	false	Visibilidad del container de notificación
notification_text	string	""	Texto que muestra la notificación
KEYS	array:string	[...]	Mapa de índice ButtonMatrix → caracter
loading_value	integer	0	Valor de la barra de progreso en SplashScreen

⚠ stored_password es la fuente de verdad del PIN. NUNCA debe ligarse a ningún widget visible. pass_current siempre comienza vacío. Son variables con responsabilidades separadas.

IX — Referencia de Expresiones EEZ Flow

Operación	Expresión correcta	Notas
División entera	<code>Math.floor(x / y)</code>	No existe operador <code>//</code> como en Python
Redondear	<code>Math.round(x)</code>	Para decimales con precisión $\times 100$
Int a string	<code>value + ""</code>	Concatenar con literal string vacío
Condicional ternario	<code>cond ? A : B</code>	Funciona en Text Expression y SPECIFIC
Longitud de string	<code>String.length(str)</code>	Retorna número de caracteres
Slice de string	<code>String.slice(str, start, end)</code>	Índices base 0
Índice de array	<code>array[index]</code>	Para el array KEYS de la ButtonMatrix
Key presionada	<code>Event.getKey(e)</code>	Solo en handler de ButtonMatrix
NOT lógico	<code>!variable</code>	Funciona para booleans
Mostrar tiempo MM:SS	<code>Math.floor(t/60)+":"+(t%60<10?"0":"")+t%60</code>	Formatea segundos como MM:SS
Mostrar decimal $\times 100$	<code>Math.floor(d/100)+"."+(d%100<10?"0":"")+d%100</code>	Muestra 234 como 2.34

✗ `String.Format()` — BUG conocido en modo Run de EEZ Studio (issue #643). No utilizar.

✗ `IntegerToString()` / `FloatToString()` — No disponibles en EEZ Flow.

✗ `transform_zoom` en texto — Causa desaparición de widgets en LVGL 8. Usar fuente más grande en `lv_conf.h`.

X — Limitaciones Conocidas de EEZ Studio + LVGL

Limitación	Impacto	Workaround
MessageBox — broken en LVGL 9	Congela toda la pantalla al presionar OK	Usar Container + Label + show_notification
Password mode no tiene Expression	No se puede cambiar en runtime	Dos TextAreas superpuestos (plain + password)
Color background sin Expression	No se puede cambiar el color por variable	Usar Estados CHECKED/DEFAULT + LVGL Widget-States
LED color via STYLE distorsiona	Red aparece como café (sistema de brillo)	Usar SPECIFIC → Color → Expression en el LED
Texto sobre Checkbox no soportado	El label del checkbox no admite posición libre	Container con Label + Checkbox (sin texto nativo)
Sin persistencia de variables	Vars se resetean al reiniciar	Requiere NVS en código C nativo
eez-flow.cpp siempre es LVGL 8	Cast incorrecto de lv_part_t	Fix manual con Ctrl+H después de cada exportación
String.Format() buggeado	No formatea correctamente	Usar concatenación + Math.floor()
transform_zoom en texto	Widgets desaparecen	Usar fuente de mayor tamaño

XI — Bitácora de Entradas de Desarrollo

#01

Pantalla negra al subir el primer sketch

DISPLAY
14 Ene

Configuración inicial de pines SPI. El backlight en GPIO 27 requería `digitalWrite(HIGH)`. `ledcWrite(0, 255)` en lugar de `ledcWrite(PIN, valor)` era el error con ESP32 Core 3.x.

#02

Selección de librería de display

DISPLAY
14 Ene

Se eligió GFX Library for Arduino (Arduino_GFX) v1.6.4. Configuración: `Arduino_ESP32SPI(DC=2, CS=15, SCK=14, MOSI=13, MISO=NOT_DEFINED)` + `Arduino_ST7789(bus, RST=-1, rotation=0, IPS=true)`.

#03

Incompatibilidad ESP32 Core 3.x — `ledcWrite`

ARDUINO
14 Ene

API cambió: `ledcWrite()` ahora espera número de PIN como primer argumento, no número de canal. `ledcSetup()` y `ledcAttachPin()` están deprecados; usar `ledcAttach()`.

#04

Colores RGB565 no declarados

ARDUINO
14 Ene

En `Arduino_GFX` 1.6.x los colores son constantes prefijadas: `RGB565_BLACK`, `RGB565_WHITE`, `RGB565_RED`. No existen como macros sueltas.

#05

Integración LVGL + Arduino_GFX — flush callback

LVGL
15 Ene

LVGL y `Arduino_GFX` no se conocen. El callback de flush actúa de traductor: recibe `lv_area_t + px_map` de LVGL y llama `gfx->draw16bitRGBBitmap()`. Requiere `lv_display_flush_ready()` al final.

#06

Estructura de carpetas: `src/` obligatoria

ARDUINO
15 Ene

Arduino IDE solo compila automáticamente `.c/.cpp` en la carpeta raíz o en `src/`. Mover todos los archivos exportados por EEZ Studio a `src/`.

#07

`lv_conf.h` — ubicación y configuración

LVGL
15 Ene

El archivo debe estar en `C:\Users\DANNY\Documents\Arduino\libraries\lv_conf.h` (nivel raíz de `libraries`, NO dentro de la carpeta `lvgl`). Cambiar `#if 0` por `#if 1` al inicio.

#08

Archivo 'nul' causa error en Windows

ARDUINO
15 Ene

`GetFileInformationByHandle` error. 'nul' es un nombre reservado de Windows. Eliminar el archivo para resolver.

#09	Touch GT911 — primer intento fallido	TOUCH 21 Ene
-----	--------------------------------------	-----------------

XPT2046_Touchscreen es para touch resistivo. La pantalla ESP32-2432S032 usa GT911 capacitivo (I2C), que requiere TAMC_GT911. Librería diferente, interface diferente, librería diferente.

#10	Instalación de TAMC_GT911	TOUCH 21 Ene
-----	---------------------------	-----------------

La librería TAMC_GT911 v1.0.2 Sí está disponible en el gestor de librerías de Arduino IDE. Buscarla como 'GT911' o 'TAMC'. NO es necesaria instalación manual desde repositorio.

#11	Coordenadas de touch invertidas en portrait	TOUCH 21 Ene
-----	---	-----------------

GT911 reporta coordenadas en landscape (320×240 internamente). Para portrait: display_x = map(raw_y, 0, 240, 0, 239); display_y = map(raw_x, 0, 320, 319, 0); con ts.setRotation(ROTATION_LEFT).

#12	lv_conf.h — #include <stdint.h> rompe .S files	LVGL 21 Ene
-----	--	----------------

El archivo lv_conf.h no debe contener #include <stdint.h> en nivel global. El archivo .S de ensamblador ARM (lv_blend_helium.S) falla al compilar. Eliminar esa línea del conf.

#13	eez-flow.cpp — cast lv_part_t	EEZ 22 Ene
-----	-------------------------------	---------------

EEZ Studio genera código LVGL 8. lv_obj_get_style_opa(obj, 0) falla en LVGL 9. Reemplazar el segundo argumento con (lv_part_t)LV_PART_MAIN en los dos patrones que genera.

#14	Fuentes Montserrat — habilitación en lv_conf.h	LVGL 22 Ene
-----	--	----------------

Las fuentes extra (10, 12, 18, 22, 34) deben habilitarse explícitamente en lv_conf.h. El error es: 'lv_font_montserrat_22' was not declared in this scope.

#15	SplashScreen con barra de progreso	EEZ 23 Ene
-----	------------------------------------	---------------

Variable loading_value (integer, 0–100) controla el valor del widget Bar. Flow: Delay 50ms → loading_value++ → IsTrue loading_value >= 100 → No: volver → Sí: cargar login_SCREEN.

#16	login_SCREEN — teclado numérico con KEYS array	EEZ 24 Ene
-----	--	---------------

ButtonMatrix con 12 botones (0-9, punto, borrar). Array KEYS mapea índice a carácter. En CLICKED: output_keyboard ← output_keyboard + KEYS[Event.getKey(e)] con casos especiales para borrado.

#17	PIN visibility toggle — dos TextAreas	EEZ 24 Ene
-----	---------------------------------------	---------------

LVGL no admite cambiar password mode en runtime. Solución: `textarea_password` (Hidden: `!eyeOpen`) + `textarea_plain` (Hidden: `eyeOpen`). Botón ojo: `eyeOpen` ← `!eyeOpen`.

#18 MENU — checkbox estado a variable

EEZ
25 Ene

El checkbox de LVGL debe ligarse via STATES → Checked → Assignable → variable. NO via expresión de Text. Este es el único binding bidireccional disponible para checkboxes.

#19 MENU — color de LED distorsionado via STYLE

EEZ
25 Ene

Asignar color via STYLE aplica el sistema de brillo de LVGL que convierte rojo en café. Solución: usar SPECIFIC → Color → Expression: `check_R ? 0xff0000 : 0x000000`

#20 MENU — display de tiempo MM:SS

EEZ
26 Ene

Expresión: `Math.floor(treatment_time/60)+":"+((treatment_time%60<10?"0":"")+treatment_time%60`

#21 MENU — cálculo de dosis con decimales

EEZ
26 Ene

LVGL no muestra floats directamente. Multiplicar por 100, almacenar como integer, mostrar con: `Math.floor(d/100)+"."+(d%100<10?"0":"")+d%100`

#22 MENU — flags DISABLED en botones play/stop

EEZ
27 Ene

PLAY Disabled: `is_running || treatment_time == 0`. STOP Disabled: `!is_running`. RESET Disabled: `is_running`. ±tiempo Disabled: `is_running`.

#23 MENU — batería: 6 LEDs con Hidden Expressions

EEZ
27 Ene

`charge_zero` Hidden: `battery_state != 0 && battery_state != 5`. `charge_quarter`: `battery_state != 1`. Etc. Solo el LED correspondiente al estado actual es visible.

#24 Container styling — bg_opa obligatorio

EEZ
28 Ene

Un container con color de fondo no lo muestra si `bg_opa` = 0 (default). Cambiar STYLE → Opacity a 255 para que el fondo sea visible.

#25 SETTINGS — widget QR Code

EEZ
29 Ene

Requiere `LV_USE_QRCODE` = 1 en `lv_conf.h`. El texto del QR se actualiza via Text Expression.

#26 CAMBIO_PASSWORD — teclado contextual con active_field

EEZ
30 Ene

Tres TextAreas. Cada uno al recibir foco: `active_field ← N`, `keyboard_visible ← true`. Al perder foco: `keyboard_visible ← false`. ButtonMatrix escribe via `SwitchCase(active_field)`.

#27 CAMBIO_PASSWORD — validación en cadena

EEZ
31 Ene

Flow de guardar: `!true pass_current == stored_password → !true pass_new == pass_confirm → !true String.length(pass_new) == 4 → stored_password ← pass_new → limpiar campos`.

#28 pngquant y Python para imágenes en EEZ

EEZ
01 Feb

EEZ Studio requiere `pngquant.exe` en PATH y Python 3.12 con `pypng + lz4`. Reiniciar EEZ después de agregar `pngquant` al PATH de Windows.

#29 Variable native stub en screens.c

EEZ
02 Feb

`output_keyboard` debe tener implementación nativa en C. Agregar stubs `get_var_output_keyboard()` / `set_var_output_keyboard()` en `screens.c`.

#30 MessageBox congela pantalla en hardware real

EEZ
03 Feb

Bug crítico: MessageBox de EEZ Studio crea un overlay modal en LVGL 9 que no se destruye al presionar OK. La pantalla queda completamente congelada. Solo se manifiesta en hardware, no en simulador.

#31 Sistema de notificación Container reemplaza MessageBox

EEZ
03 Feb

Solución al #30: Container con Label, Hidden Expression `!show_notification`. Color por States (`CHECKED=verde`, `DEFAULT=rojo`). Flow: `set_notification_text → show_notification=true → Delay → show_notification=false`.

#32 Fix login — limpiar PIN después de validación

EEZ
04 Feb

Agregar `output_keyboard ← ""` al final de los flows de PIN correcto e incorrecto. Sin esto, el campo queda con el PIN anterior en el siguiente intento.

#33 Prueba en hardware — compilación exitosa

ARDUINO
05 Feb

Compilación sin errores aplicando todos los fixes documentados. Flash correcto. UI visible y funcional en pantalla real. Touch respondiendo correctamente.

#34 Configuración VS Code IntelliSense

VSCODE
06 Feb

Configurados `arduino.json` (board `esp32:esp32:esp32`, port `COM3`, output `../build`) y `c_cpp_properties.json` con rutas al compilador xtensa y todos los includes de librerías.

#35

Optimización velocidad compilación con VS Code**VSCODE**
07 Feb

La carpeta de build ../build reutiliza archivos objeto entre compilaciones. Reducción notable del tiempo vs Arduino IDE que recompila todo desde cero.

#36

Pendiente: integración hardware (batería, LEDs, NVS)**GENERAL**
08 Feb

La UI está completa. Próximos pasos: leer ADC para battery_state, controlar LEDs físicos, persistir stored_password en NVS, comunicación UART con microcontrolador principal.

XII — Estado del Proyecto

Componente	Estado	Notas
SplashScreen	Completado	Animación de carga funcional
login_SCREEN	Completado	PIN, toggle visibilidad, notificaciones sin MessageBox
MENU	Completado	LEDs, temporizador, dosis, batería
SETTINGS	Completado	QR code funcional
CAMBIO_PASSWORD	Completado	Validación completa del PIN
Compilación en hardware	Completado	Sin errores, flash exitoso
Touch en hardware	Completado	GT911 con coordenadas portrait corregidas
Lectura ADC batería	Pendiente	Variable battery_state se actualiza manualmente por ahora
Control físico de LEDs	Pendiente	Checkboxes listos, GPIOs pendientes de mapear
Persistencia NVS	Pendiente	stored_password se resetea al reiniciar
Comunicación UART/I2C	Pendiente	Protocolo con MCU principal por definir